DESIGN AND VERIFICATION OF SYNCHRONOUS FIFO USING VERILOG & UVM

A Learning-Based Project Report

Submitted By:

Mohammad Rehan
BT23EC003

Course:

Digital Design and Verification using SystemVerilog & UVM

Tools & Technologies:

Xilinx Vivado, SystemVerilog, UVM Methodology

Department : Electronics and Communication Engineering

(National Institute of Technology, Mizoram**)**

Academic Year:

2025 – 2026

Declaration:

This project has been developed purely for learning and academic purposes. It is not intended for commercial or industrial deployment.

Abstract

This report presents the design and verification of a parameterized synchronous FIFO (First-In-First-Out) buffer implemented using SystemVerilog in Xilinx Vivado. The FIFO is designed to support configurable data width and depth, enabling scalability for different digital system requirements. Key status flags including full, empty, almost full, and almost empty are generated to monitor buffer conditions, while overflow and underflow detection logic improves operational safety.

The design follows a modular architecture consisting of separate control and memory modules, which simplifies design understanding, improves maintainability, and supports future expansion. The control unit manages write and read pointers, count logic, and status flag generation, while the memory block stores data sequentially according to FIFO principles.

Verification is performed using a structured SystemVerilog testbench containing generator, driver, monitor, and scoreboard components connected through mailbox-based communication. Functional simulation validates correct write, read, simultaneous operations, and error conditions. Synthesis and timing analysis confirm low FPGA resource utilization, successful timing closure, and reliable implementation suitability for FPGA-based digital systems.

---

1. Introduction

This project presents the design and verification of a parameterized synchronous FIFO (First-In First-Out) using SystemVerilog. The design is implemented using a modular approach consisting of control logic and memory unit.

The verification environment is developed using SystemVerilog-based components such as generator, driver, monitor, and scoreboard. A mailbox-based communication mechanism is used between components to ensure proper data flow and synchronization.

The scoreboard is used to validate the correctness of FIFO operations by comparing expected and actual data. The entire system is simulated to verify functionality such as read, write, full, empty, overflow, and underflow conditions.

FIFO addresses these challenges by:

- Providing temporary data storage
- Maintaining proper data sequencing
- Enabling smooth data transfer between producer and consumer

This project implements a **synchronous FIFO**, where both read and write operations are controlled by a single clock, ensuring simple and reliable timing behavior.

---

2. Why FIFO is Important

Without a FIFO:

- A fast producer can overflow a slow consumer by generating data faster than it can be processed.
- A slow producer can cause starvation for a fast consumer by not supplying data quickly enough.

A FIFO solves this problem by acting as an intermediate buffer, allowing data to be temporarily stored and transferred safely between modules operating at different speeds.

---

3. Applications

FIFO is widely used in:

- UART communication systems
- AXI-based data buffering
- DSP pipelines

- DMA transfers
- Packet buffering in networking systems
- Streaming data interfaces

---

4. Types of FIFO

There are two main types of FIFO:

1. **Synchronous FIFO**
    - Uses a single clock for read and write
    - Simpler design and easier timing control
2. **Asynchronous FIFO**
    - Uses separate clocks for read and write
    - Requires complex synchronization (e.g., Gray code pointers)

This project focuses on **Synchronous FIFO design**.

---

5. Project Objective

The objectives of this project are:

- To design a **parameterized FIFO**
- To implement **read and write control logic**
- To generate **status flags (full, empty, almost flags)**
- To detect **overflow and underflow conditions**
- To verify functionality using SystemVerilog testbench
- To analyze synthesis and timing reports

---

6. FIFO Architecture :

The FIFO design is modular and consists of:

- fifo_top (Top-level module) → connects control + memory
- fifo_ctrl (Control logic) → handles pointers, count, flags
- fifo_mem (Memory block) → stores data

Parameterized design (WIDTH, DEPTH)

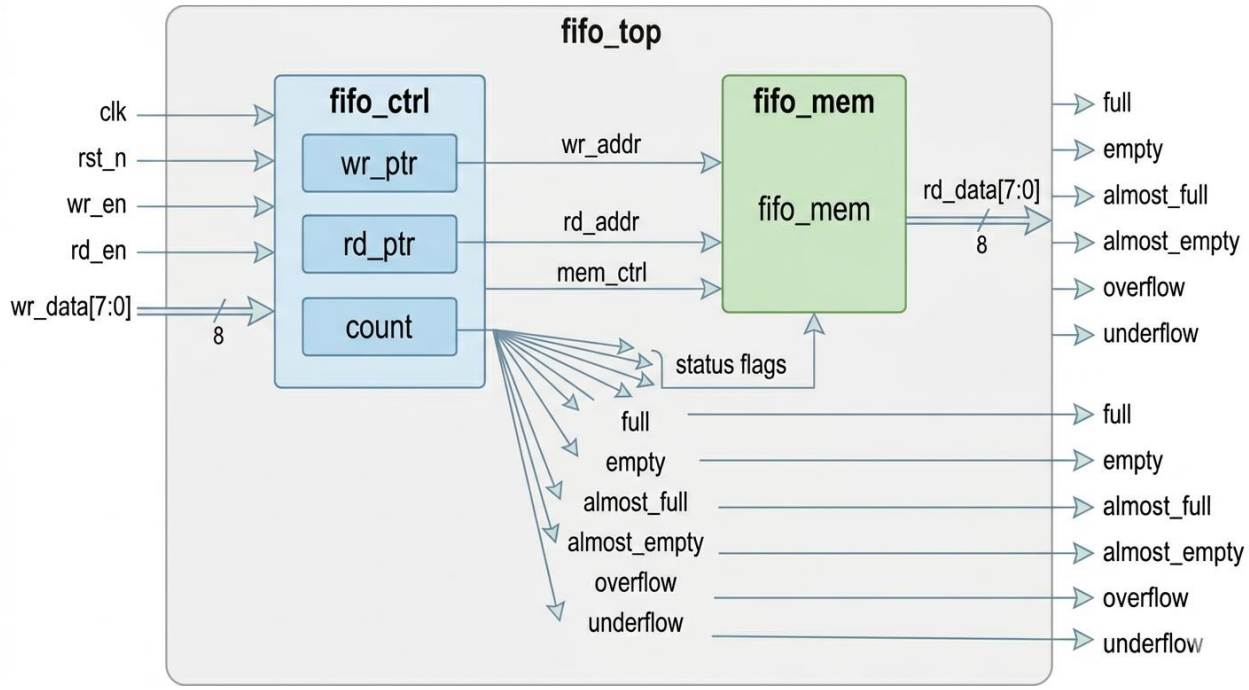

Figure 1: Block Diagram of Synchronous FIFO Architecture

This diagram shows the overall architecture of the FIFO system, including the control unit, memory block, and data flow between them. The control unit manages read/write operations using pointers and a counter.

---

7. FIFO Operation

Write Operation

Condition:

- wr_en && !full

Action:

- Data is written into memory
- Write pointer increments
- Count increases

Read Operation

- Condition:
- rd_en && !empty

Action:

- Data is read from memory
- Read pointer increments
- Count decreases

Simultaneous Read/Write

- Both pointers increment
- Count remains unchanged
- Ensures continuous data flow

---

8. Memory Design

logic [7:0] mem [0:15];

- Data width: 8 bits
- Depth: 16 entries

9. Status Flags

Flag	Condition
Full	count == DEPTH
Empty	count == 0
Almost Full	count == DEPTH - 1
Almost Empty	count == 1

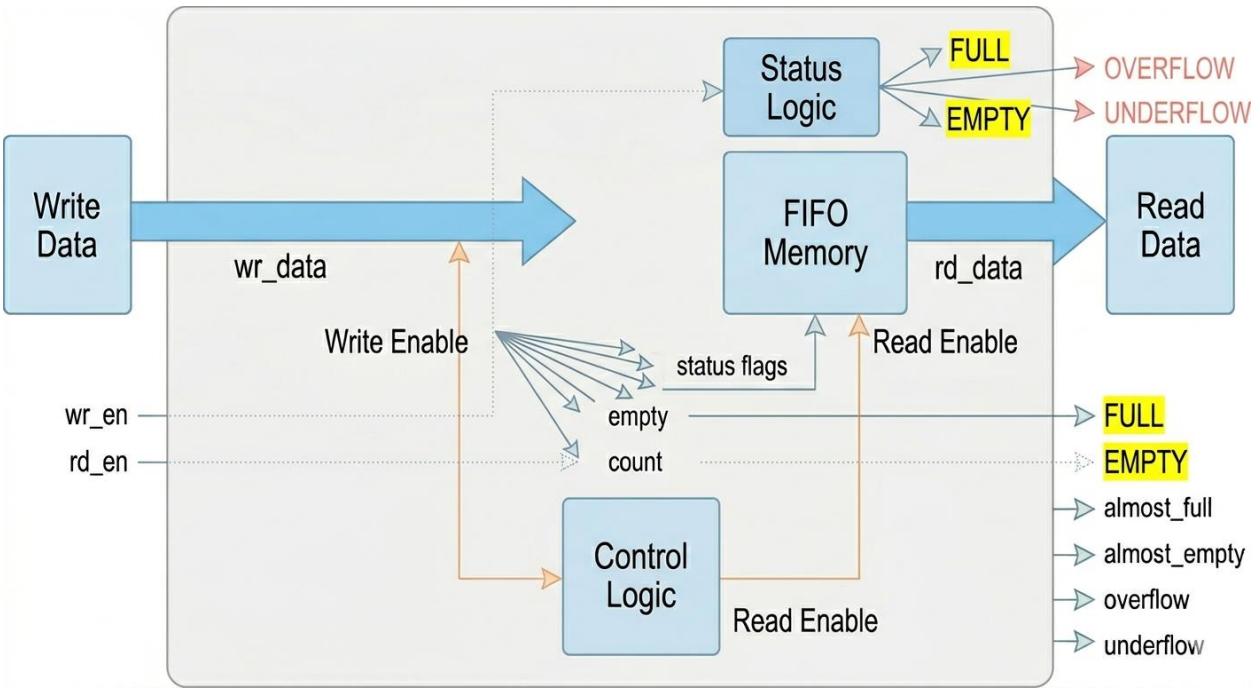

Figure 2: FIFO Status and Error Condition Logic

This diagram represents the generation of status signals and error conditions. Overflow occurs when writing to a full FIFO, while underflow occurs when reading from an empty FIFO.

10. Safety Features :

Overflow Protection

Assertion:

assert property (@(posedge clk) !(wr_en && full))

Underflow Protection

Assertion:

assert property (@(posedge clk) !(rd_en && empty))

This improves verification quality.

---

11. Design Challenges

- Preventing overflow and underflow
- Maintaining correct pointer synchronization
- Ensuring correct behavior during simultaneous operations
- Preserving data integrity

---

12. Verification Architecture

- Interface (fifo_if) → connects DUT & TB
- Transaction (fifo_txn) → defines data packet
- Generator → creates random transactions
- Driver → drives signals to DUT
- Monitor → observes DUT signals

- Scoreboard → checks correctness

The verification environment follows a structured approach similar to UVM methodology.

SCOREBOARD IMPLEMENTATION

The scoreboard is implemented using a queue data structure to store expected values. During write operations, data is pushed into the queue, and during read operations, data is popped and compared with the actual output from the FIFO.

If the expected and actual values match, a PASS message is displayed; otherwise, a FAIL message is reported. This ensures functional correctness of the FIFO design.

DATA FLOW :

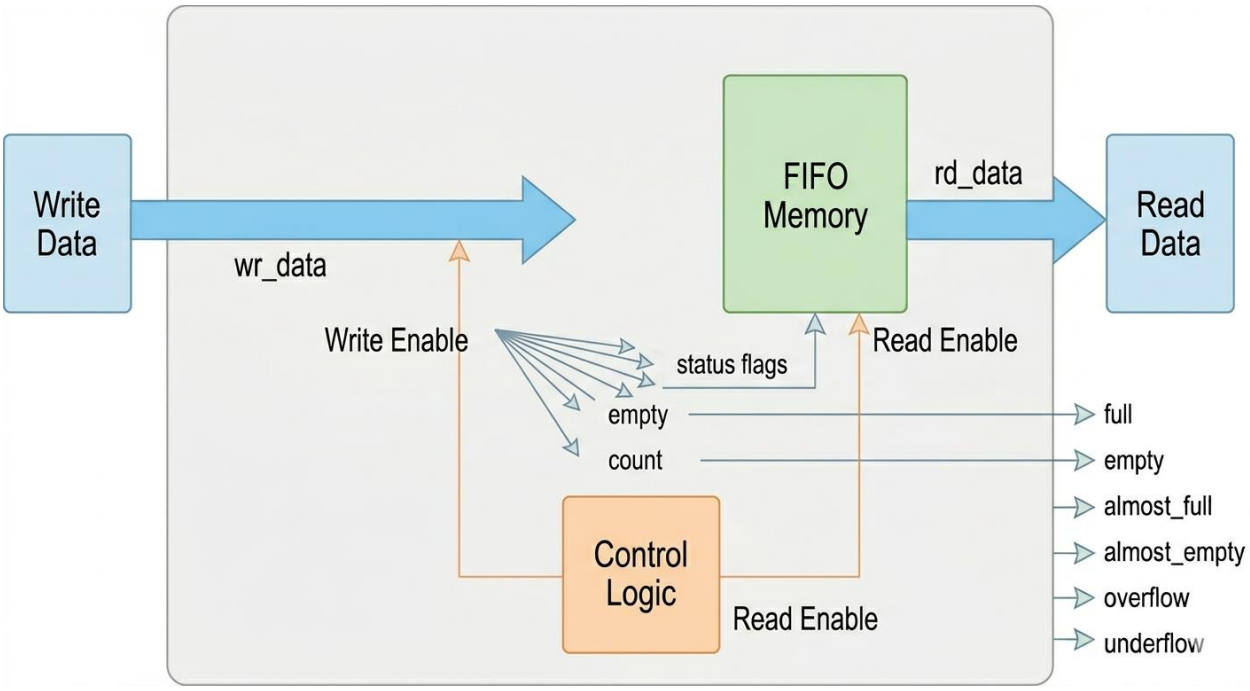

Figure 3: Data Flow Diagram of FIFO System

The data flow diagram illustrates how data is written into the FIFO memory and read out based on control signals such as write enable and read enable.

Generator → Driver → DUT → Monitor → Scoreboard

- Generator creates data
- Driver sends data
- DUT processes
- Monitor captures
- Scoreboard verifies

---

13. Simulation Results Analysis

From waveform:

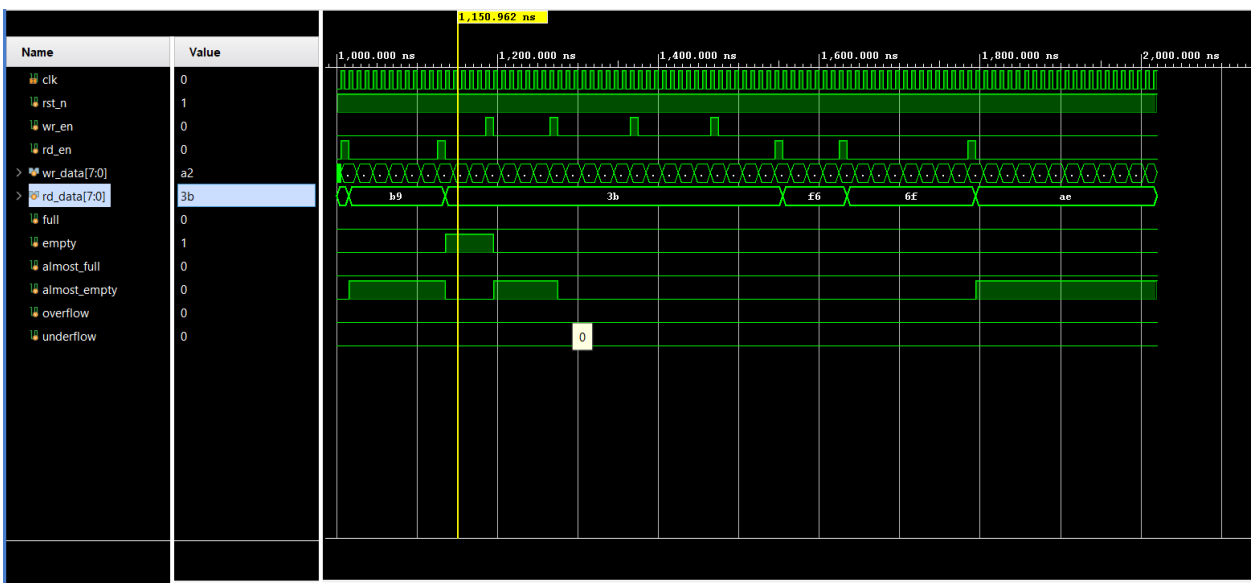

Figure 4: Complete FIFO Functional Simulation Showing Write, Read, Data Flow, and Status Flag Verification

The simulation waveform verifies correct FIFO behavior:

- Write and read operations are clearly observed
- Data is read in the same order as written
- Status flags transition correctly

Example:

Written Data: 69

Read Data: 69

This confirms that FIFO maintains correct data ordering and functionality.

---

14. Timing Report Analysis

Timing Analysis :

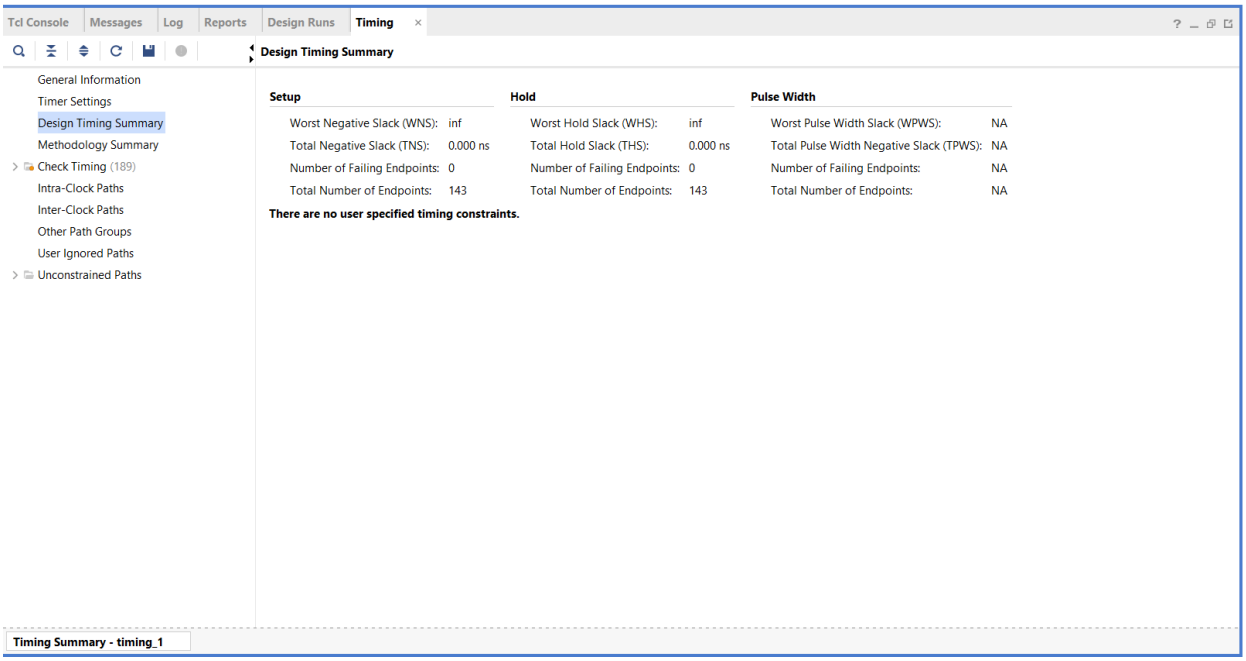

Figure 5: Design Timing Summary Report

- Worst Negative Slack (WNS): ∞
- Total Hold Slack (THS): 0.000 ns
- Failing Endpoints: 0

No setup or hold violations

Design is timing safe

---

15. Utilization Report Analysis

From utilization:

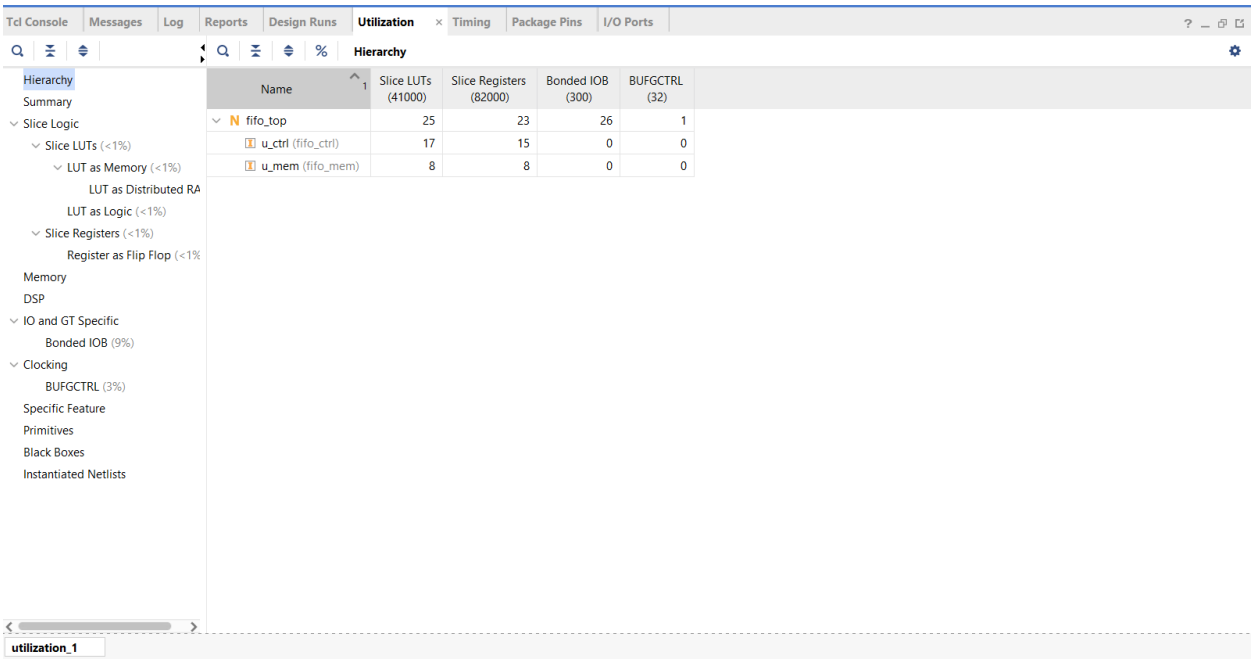

Figure 6: FPGA Resource Utilization Report

LUT Usage

25 LUTs

Very low usage.

Registers

23 Registers

Efficient sequential design.

IO Usage

26 IO Ports

This shows lightweight FPGA-friendly implementation.

16. Synthesized Design Analysis

Netlist confirms:

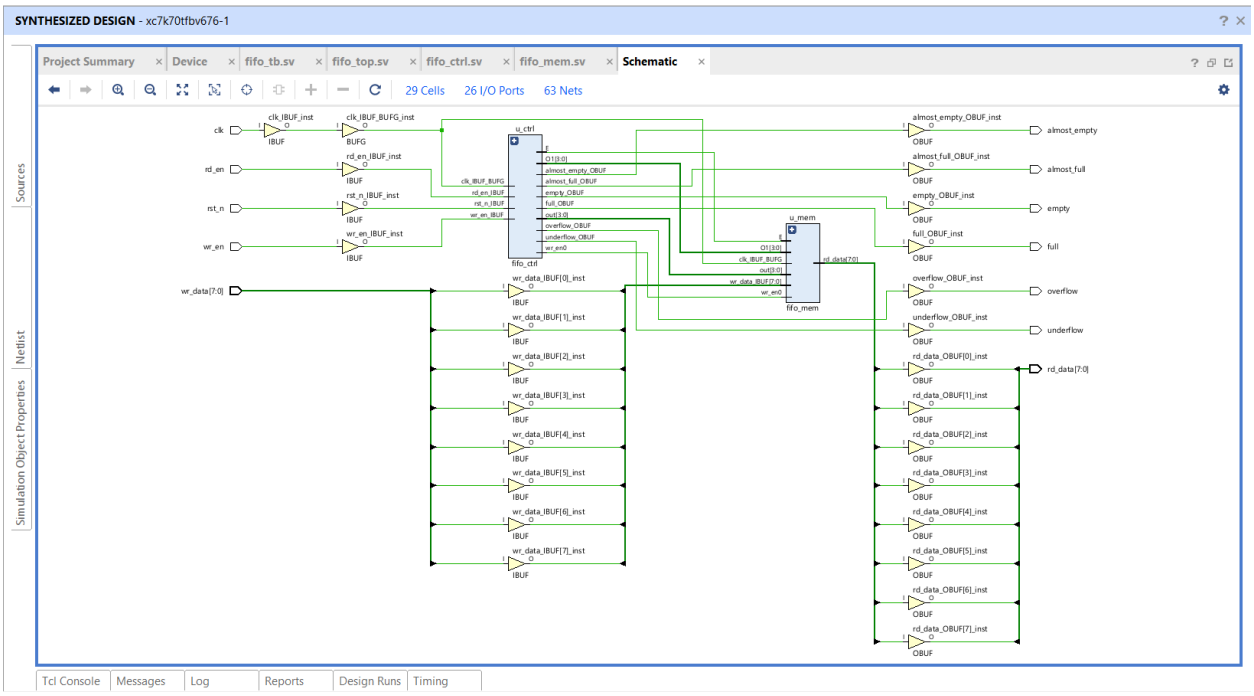

Figure 7: Synthesized RTL Schematic of Parameterized FIFO Design

The synthesized RTL shows:

- Proper module hierarchy
- Clear separation of control and memory
- Optimized data path

17. Advantages

- Simple and efficient design
- Parameterized and scalable
- Low resource utilization

- Suitable for real-time applications

---

18. Conclusion

The parameterized FIFO design was successfully implemented and verified using SystemVerilog in Vivado. The design ensures correct data sequencing, efficient handling of full and empty conditions, and reliable protection against overflow and underflow scenarios. Simulation results confirm functional correctness, while synthesis reports demonstrate efficient resource utilization and successful timing closure. The design is suitable for FPGA-based applications and can be extended for advanced use cases.

---

19. Future Scope

- Implementation of asynchronous FIFO
- Multi-clock domain support
- Integration with AXI protocol
- High-speed data buffering systems